

Guia de Estudo — Prova 01

Paradigmas de Linguagens de Programação
Capítulos 1 a 5 de Sebesta

Aluno: Victor Hugo Pacchioni Sgobbi

RA: 24000732-2

Bibliografia: SEBESTA, R. W. *Conceitos de Linguagens de Programação*, 11. ed.

Escopo: fundamentos, evolução, descrição, análise e variáveis

*Anotações próprias, montadas a partir dos slides de revisão do professor
e das atividades que resolvi ao longo do bimestre.*

Como eu montei e como eu uso este guia

Escrevi este material reorganizando os slides da revisão por **mecanismo**, não por ordem de aula. A meta que o professor deixou explícita é que a resposta boa tem três partes, e foi nesse formato que eu treinei cada tópico aqui:

FÓRMULA DA RESPOSTA

conceito correto + mecanismo (como funciona) + consequência (o que muda na prática). Só definir o termo não fecha a questão; só dar exemplo, também não.

Cada capítulo abaixo segue a mesma estrutura: os conceitos, uma tabela de comparação onde há risco de confundir, um exemplo trabalhado e um bloco de *armadilha* com o erro que eu cometeria se fosse no automático. No fim há um autoteste e o checklist de 10 minutos.

As cinco lentes

Os cinco capítulos não são assuntos soltos: são cinco perguntas sobre a mesma linguagem.

Lente	Capítulo	Pergunta que responde
Projeto	1 — Fundamentos	Como avaliar e o que se ganha ou se perde em cada escolha?
História	2 — Evolução	De onde veio e a que domínio respondia?
Descrição	3 — Sintaxe e semântica	Como especificar formalmente forma e significado?
Análise	4 — Léxico e parser	Como uma ferramenta reconhece se o programa pertence à linguagem?
Execução	5 — Variáveis	Como nomes ganham tipo, memória, valor e tempo de vida?

CONEXÃO QUE AMARRA TUDO

A **descrição formal orienta o parser** — a gramática do capítulo 3 é literalmente a entrada do analisador do capítulo 4. E as **decisões sobre tipos e memória** do capítulo 5 são o que materializa os critérios de **confiabilidade e custo** do capítulo 1. Se eu conseguir contar essa cadeia de ponta a ponta, mostrei que entendi o conjunto e não só as partes.

O que eu preciso conseguir fazer

Verbo	Aplicado a
Comparar	critérios de avaliação, paradigmas e estratégias de implementação
Relacionar	linguagens, seus domínios e suas contribuições
Aplicar	gramática, derivação e classificação de símbolos
Explicar	léxico, parser, LR e as abordagens de semântica
Analisar	vinculação, tempo de vida, escopo e previsão de saída

Roteiro

1. Capítulo 1 — Fundamentos: critérios, influências, categorias e implementação

2. Capítulo 2 — Evolução: coexistência, domínios e contribuições

3. Capítulo 3 — Descrição: FNB, derivação, ambiguidade e semântica

4. Capítulo 4 — Análise: léxico, parser, recursão à esquerda e LR

5. Capítulo 5 — Variáveis: atributos, vinculação, tempo de vida e escopo

6. Armadilhas que eu preciso evitar

7. Autoteste com resposta curta

8. Checklist final de 10 minutos

1. Fundamentos — por que estudar e como avaliar

1.1 Por que estudar conceitos de linguagens

A justificativa não é decorar sintaxe de linguagem nenhuma. São seis ganhos concretos:

Ganho	O que muda na prática
Expressar	Amplia o repertório para formular soluções — quem só conhece uma linguagem pensa só do jeito dela.
Escolher	Permite casar linguagem com o domínio do problema em vez de usar sempre a mesma.
Aprender	Reconhecer conceitos conhecidos numa linguagem nova encurta muito a curva.
Implementar	Entender o que compilador e <i>runtime</i> fazem explica custo e comportamento.
Usar melhor	Explorar com consciência recursos da linguagem que já se usa.
Evoluir	Avaliar tendências por mérito técnico, não só por popularidade.

Os domínios que importam nessa escolha: **científico, comercial, inteligência artificial, sistemas e Web**.

1.2 Os quatro critérios de avaliação

Este é o tópico mais provável de cair pedindo comparação. Os quatro critérios e seus sub-critérios:

Critério	Definição	Sub-critérios
Legibilidade	Facilidade de ler e compreender um programa escrito por outra pessoa.	simplicidade geral, ortogonalidade, tipos de dados, projeto da sintaxe
Capacidade de escrita	Facilidade de expressar a solução na linguagem.	simplicidade, ortogonalidade, abstração, expressividade, adequação ao domínio
Confiabilidade	O programa cumprir a especificação em todas as condições.	verificação de tipos, tratamento de exceções, poucos <i>aliases</i> , e ainda legibilidade e escrita
Custo	Custo do ciclo de vida inteiro .	treinamento, escrita, compilação, execução, implementação da linguagem, manutenção

ORTOGONALIDADE

Um conjunto pequeno de construções primitivas que podem ser **combinadas de poucas maneiras e sem exceções**. Alta ortogonalidade ajuda legibilidade e escrita ao mesmo tempo, porque não obriga a memorizar casos especiais. Ortogonalidade **demais**, porém, permite combinações estranhas e pode prejudicar a legibilidade — ela não é um bem infinito.

1.3 Os trade-offs (o ponto que o professor destacou)

A pergunta clássica é armadilha se eu responder que um critério é "melhor". Eles se tensionam:

- **Confiabilidade × custo de execução:** mais checagens em tempo de execução aumentam a confiabilidade e também o custo. Verificação de índice de vetor é o exemplo canônico.
- **Escrita × legibilidade:** mais concisão e mais operadores facilitam escrever e dificultam ler. APL é o caso extremo.
- **Flexibilidade × detecção antecipada:** tipagem dinâmica dá flexibilidade e joga os erros para a execução; tipagem estática detecta cedo e restringe.

COMO EU FECHO ESSA QUESTÃO

Sempre com: "**não existe melhor linguagem fora de contexto**" — a escolha depende do domínio e de qual critério pesa mais ali. Em software embarcado crítico, confiabilidade domina custo de execução; em um script descartável, capacidade de escrita domina tudo.

1.4 Influências sobre o projeto de linguagens

Influência	Efeito
Arquitetura de von Neumann	Memória + processador separados, execução sequencial. Favorece variáveis, atribuição e iteração — é a origem estrutural das linguagens imperativas , e a razão de iteração ser mais eficiente que recursão nelas.
Metodologias de software	Programação estruturada → abstração de dados → orientação a objetos → concorrência. O projeto de software pressiona o projeto da linguagem, que responde com novas construções.

1.5 Categorias de linguagens

Categoria	Modelo	Exemplos
Imperativa	Variáveis, atribuição, iteração; espelha von Neumann.	C, Fortran, COBOL
Funcional	Aplicação de funções a parâmetros; sem estado mutável no núcleo.	Lisp, Scheme, ML
Lógica	Regras e fatos; baseada em regras, ordem de execução não é do programador.	Prolog
Marcação / programação híbrida	Marcação estendida com construções de programação.	JSTL, XSLT, PHP

Orientação a objetos não é uma quinta categoria isolada: em geral aparece como uma extensão da imperativa (Java, C++) e linguagens reais **combinam** ideias de mais de uma categoria.

1.6 Métodos de implementação

Método	Como funciona	Compromisso
Compilação	Traduz o fonte para linguagem de máquina antes da execução.	Execução rápida; ciclo de desenvolvimento mais lento.
Interpretação pura	Um interpretador executa o fonte instrução a instrução , sem tradução prévia.	Depuração fácil e flexível; execução de 10 a 100× mais lenta e maior consumo de memória.
Híbrida	Traduz para uma forma intermediária portátil , que uma VM interpreta.	Meio-termo entre portabilidade e velocidade. Java <i>bytecode</i> é o exemplo.
JIT	Começa na forma intermediária e compila para máquina durante a execução os trechos frequentes, reutilizando-os depois.	Ganha velocidade sem perder portabilidade; custa aquecimento e memória.

PISTA DE PROVA

Enunciado que disser "**compila em tempo de execução e reutiliza o código gerado**" é JIT, não compilação e não híbrida. O que distingue o JIT da híbrida pura é exatamente o **reúso** do trecho já

compilado.

As fases do compilador, na ordem: **análise léxica** → **análise sintática** → **análise semântica** → **geração de código intermediário** → **otimização** → **geração de código**. As duas primeiras são o capítulo 4.

2. Evolução — coexistência, não substituição

2.1 A ideia central

A genealogia das linguagens é uma **rede**, não uma escada de progresso. O ciclo que o slide resume é: **necessidade** → **ideia** → **influência** → **adaptação** → **coexistência**.

COMO EU FORMULO

Uma linguagem pode **continuar útil no seu domínio** e, ao mesmo tempo, **transmitir** sintaxe, paradigma ou estratégia de implementação para projetos posteriores. Fortran ainda roda computação científica e COBOL ainda roda sistemas bancários décadas depois de surgirem alternativas. **Nova ≠ superior; antiga ≠ irrelevante.**

Dois fatores que fazem uma linguagem influenciar outra sem substituí-la: **a inércia da base instalada** (custo de reescrever sistemas que funcionam) e **a especialização de domínio** (a linguagem antiga continua sendo a melhor ferramenta para aquele problema específico).

2.2 As quatro contribuições que caem sempre

Linguagem	Domínio	Contribuição
Fortran (1957)	Computação científica	Primeira de alto nível amplamente usada; eficiência numérica e tradução de fórmulas. Provou que compilador podia competir com código de máquina escrito à mão.
Lisp (1959)	Processamento simbólico e IA	Funções, listas e recursão; funda a tradição funcional . Tipagem dinâmica e gerenciamento automático de memória.
COBOL (1960)	Processamento comercial	Dados e registros de negócio; sintaxe verbosa em inglês pensada para legibilidade por gestores. Herdeira do FLOW-MATIC de Grace Hopper.
Smalltalk (1972–80)	Orientação a objetos	Primeira linguagem OO plena: tudo é objeto, comunicação por mensagens , ambiente integrado e interface gráfica.

2.3 Panorama por domínio

Linguagem / família	Problema ou domínio	Leitura principal
Plankalkül (1945)	Ideias de alto nível, nunca implementada na época	pioneirismo
Short Code, Speedcoding, A-0	Pseudocódigos interpretados	interpretação inicial
Fortran / COBOL	Científico / comercial	imperativo e domínio
ALGOL 60	Descrição formal de algoritmos	estrutura de blocos e BNF
Lisp / Scheme	Símbolos e funções	funcional
APL / SNOBOL	Arrays / strings	dinamismo e notação
Prolog / Smalltalk	Regras / objetos	lógico / orientado a objetos
Java / Lua	VM com JIT / scripting embarcado	estratégias híbridas
JSTL / XSLT	Documentos e regras	marcação + programação

2.4 ALGOL 60: por que importa mesmo sem ter sido muito usada

Contribuições que se tornaram padrão: **estrutura de blocos**, **escopo estático**, passagem de parâmetros por valor e por nome, recursão, e — decisivo para o capítulo 3 — foi a **primeira linguagem descrita formalmente por uma gramática BNF**. Ela falhou em adoção (sem E/S padronizada, sem apoio da IBM) e venceu em influência: C, Pascal, Java e praticamente toda a linhagem imperativa descendem dela.

ARMADILHA DO CAPÍTULO 2

Tratar a lista de linguagens como **cronologia a decorar**. O que a questão pede é **relação**: linguagem → domínio → contribuição → quem herdou. Data isolada quase nunca vale ponto; "Lisp deu origem à tradição funcional que reaparece em ML, Haskell e nos recursos funcionais de Java e Python" vale.

3. Descrição — sintaxe, gramática e semântica

3.1 Vocabulário que não pode ser confundido

Termo	Definição	Exemplo em <code>total = 7;</code>
Sintaxe	Forma e organização das construções.	a ordem <i>identificador, igual, número, ponto e vírgula</i>
Semântica	Significado e efeito das construções.	armazenar 7 na variável <code>total</code>
Lexema	A ocorrência concreta reconhecida no texto.	<code>total</code> , <code>=</code> , <code>7</code> , <code>;</code>
Token	A categoria do lexema, entregue ao parser.	<code>IDENT</code> , <code>ATRIB</code> , <code>LITERAL</code> , <code>DELIM</code>
Sentença	Cadeia formada só por terminais , derivável da gramática.	a linha inteira já derivada
Linguagem	O conjunto de todas as sentenças válidas.	todos os programas legais

NÃO CONFUNDIR

Lexema é o trecho reconhecido; token é a classificação. Vários lexemas diferentes (`total` , `soma` , `n`) compartilham o mesmo token `IDENT` . Trocar os dois é o erro mais fácil de cometer sob pressão.

Reconhecedores × geradores

Duas formas de definir a mesma linguagem: o **reconhecedor** lê uma cadeia e responde se ela pertence à linguagem (é o parser); o **gerador** produz sentenças válidas (é a gramática). São visões complementares do mesmo conjunto.

3.2 FNB: anatomia de uma gramática

```
<atr> ::= <id> "=" <expr> ";"
<expr> ::= <num> | <num> "+" <num>
<id> ::= "a" | "b"
<num> ::= "1" | "2"
```

Elemento	O que é	No exemplo
Terminais	Símbolos que aparecem na sentença final; não se expandem.	<code>a</code> <code>b</code> <code>1</code> <code>2</code> <code>=</code> <code>+</code> <code>;</code>
Não terminais	Abstrações que ainda precisam ser expandidas.	<code><atr></code> <code><id></code> <code><expr></code> <code><num></code>
Produção	Regra que define as alternativas de um não terminal.	cada linha; <code> </code> significa "ou"
Símbolo inicial	Não terminal por onde toda derivação começa.	<code><atr></code>

3.3 Derivação completa (exemplo trabalhado)

Derivando `b = 1 + 2 ;` pela gramática acima, sempre **mais à esquerda**:

- `<atr>`
- `<id> = <expr> ;` aplicando `<atr> ::= <id> "=" <expr> ";"`
- `b = <expr> ;` aplicando `<id> ::= "b"`
- `b = <num> + <num> ;` aplicando `<expr> ::= <num> "+" <num>`

```
5. b = 1 + <num> ;           aplicando <num> ::= "1"
6. b = 1 + 2 ;               aplicando <num> ::= "2" ← sentença
```

TESTE DE CHEGADA

Os passos 1 a 5 são **formas sentenciais**: ainda contêm não terminais. O passo 6 é **sentença**: só terminais. A derivação termina quando não sobra nenhum não terminal — esse é o critério que eu uso para saber se posso parar.

3.4 Árvore de análise, ambiguidade e precedência

A **árvore de análise** mostra a estrutura hierárquica da derivação: a raiz é o símbolo inicial, os nós internos são não terminais e as folhas são terminais.

Uma gramática é **ambígua** quando uma mesma sentença admite **duas árvores distintas** — ou seja, dois agrupamentos e, portanto, dois significados:

```
1 - 2 / 3      →      (1 - 2) / 3  ≠  1 - (2 / 3)
```

Ambiguidade é um problema real porque o significado passa a depender do parser, não da especificação.

Como desambiguar

Organizando a gramática em **níveis**, um por faixa de precedência. O operador de menor precedência fica no nível mais alto:

```
<expr> ::= <expr> "+" <termo> | <termo>
<termo> ::= <termo> "*" <fator> | <fator>
<fator> ::= "(" <expr> ")" | <id> | <num>
```

Como ***** está mais fundo na árvore, ele é agrupado primeiro — **precedência**. E a recursão **à esquerda** (**<expr>** aparece à esquerda na própria produção) dá **associatividade à esquerda**. Isso liga direto com o capítulo 4: essa forma é ótima para descrever precedência e **péssima** para um parser descendente recursivo.

3.5 FNB estendida (FNBE)

Notação	Significa
[item]	opcional — zero ou uma ocorrência
{ item }	repetição — zero ou mais ocorrências
(a b)	agrupamento com alternativa

A FNBE não aumenta o poder de expressão — descreve as mesmas linguagens livres de contexto —, só torna a gramática mais compacta e legível, dispensando produções auxiliares recursivas.

3.6 As três abordagens de semântica

Abordagem	Base	Como descreve o significado	Uso típico
Operacional	Execução	Pelas mudanças de estado em uma máquina abstrata ou real, passo a passo.	manuals de linguagem, ensino
Denotacional	Função matemática	Mapeia cada construção para um objeto matemático e define funções sobre ele.	o mais rigoroso; base para prova de propriedades

Axiomática	Lógica	Usa asserções, pré e pós-condições e regras de inferência.	verificação formal de corretude
-------------------	--------	---	---------------------------------

A OUTRA DIVISÃO QUE CAI

Semântica estática: restrições verificáveis **sem executar** — compatibilidade de tipos, variável declarada antes do uso. Verificada em compilação.

Semântica dinâmica: o significado **durante a execução** — o efeito real de cada construção.

Atenção: apesar do nome, a semântica **estática** tem mais a ver com forma do que com significado, e é por isso que ela é verificável por gramáticas de atributos.

4. Análise léxica e sintática

4.1 Divisão de trabalho

Léxico e parser **cooperam, não duplicam**. Para a entrada `soma = n + 10 ; :`

soma	=	n	+	10	;
IDENT	ATTRIB	IDENT	OP	LITERAL	DELIM

Componente	Enxerga	Baseia-se em	Entrega
Analisador léxico	Caracteres	Padrões (expressões regulares)	Uma sequência de tokens
Parser	Tokens	A gramática livre de contexto	Árvore de análise, ou erro sintático

Por que separar? Três razões: **simplicidade** (o léxico resolve detalhes de caractere que poluiriam a gramática), **eficiência** (o léxico pode ser otimizado isoladamente e consome boa parte do tempo) e **portabilidade** (a parte dependente de codificação de caracteres fica confinada no léxico).

4.2 Descendente × ascendente

	Descendente (top-down)	Ascendente (bottom-up)
Direção	Da raiz para as folhas	Das folhas para a raiz
Operação	Expande produções	Reduz até o símbolo inicial
Derivação	Mais à esquerda	Inverso da derivação mais à direita
Implementações	Descendente recursivo, LL	LR, SLR, LALR
Gramáticas aceitas	Menos — exige ajustes	Mais — classe maior de gramáticas
Legibilidade	Código espelha a gramática; fácil escrever à mão	Tabelas geradas por ferramenta; difícil à mão

FRASE QUE RESUME

As duas abordagens percorrem **a mesma estrutura em direções opostas** e chegam ao mesmo veredito sobre a sentença. A diferença está em *quando* cada uma decide qual produção usar.

4.3 Lookahead e recursão à esquerda

O parser descendente precisa escolher **qual produção aplicar** antes de consumir a entrada. O **lookahead** — olhar o próximo token — é o que torna essa escolha possível sem retrocesso.

O problema fatal é a **recursão à esquerda**:

$$E \rightarrow E + T \mid T$$

Para expandir `E`, o parser chama `E` de novo **antes de consumir qualquer entrada**. Nada muda entre uma chamada e a seguinte: **recursão infinita** e estouro de pilha. A solução é reescrever eliminando a recursão à esquerda:

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \epsilon \end{aligned}$$

Agora **E** consome **T** primeiro, e só então decide se continua. A entrada sempre avança, então a recursão termina.

REGRA PRÁTICA

Gramáticas para **análise descendente** precisam ser ajustadas para **eliminar recursão à esquerda**. Parsers **ascendentes** (LR) lidam com ela sem problema — inclusive preferem, porque a recursão à esquerda dá associatividade à esquerda naturalmente. É por isso que a gramática "bonita" do capítulo 3 não serve direto para um descendente recursivo.

4.4 Parser LR: estado + símbolo = decisão

O LR mantém uma **pilha** que alterna estados e símbolos já reconhecidos, e consulta duas tabelas:

Tabela	Indexada por	Decide
ACTION	[estado no topo, terminal lido]	a ação: shift (empilhar e avançar), reduce (trocar o lado direito de uma produção pelo esquerdo), accept ou erro
GOTO	[estado, não terminal]	o novo estado depois de uma redução

pilha: 0 id 5 + 8 entrada restante: num \$

ACTION[8, num] = shift → empilha num e o estado correspondente

ACTION[s, \$] = accept → sentença aceita

Em uma frase: **ACTION decide a ação, GOTO conduz a estrutura já reduzida**. Chegar em **accept** com a entrada esgotada significa que a sentença pertence à linguagem.

Limite honesto do experimento que fizemos em aula: a tabela vinha pronta. A atividade observa o **uso** da tabela, não a sua construção.

4.5 Erros: léxico ou sintático?

Erro	Natureza	Por quê
Texto entre aspas não finalizado	Léxico	Falha na formação de um elemento — o token nunca se fecha.
Ponto e vírgula ausente	Sintático	Cada token é válido; a sequência é que não casa com a gramática.
Parêntese não fechado	Sintático	Estrutura incompleta — o parser esperava um terminal e não o encontrou.
Tipos incompatíveis	Semântico (estático)	Forma correta, significado inválido — passa do parser e cai na análise semântica.

Critério que eu uso: o erro está na **formação de um único token** (léxico) ou no **arranjo de tokens bem formados** (sintático)?

5. Variáveis — atributos, vinculação, tempo de vida e escopo

5.1 A variável e seus seis atributos

Uma variável é uma **abstração de uma célula de memória**, caracterizada por seis atributos:

Atributo	O que é
Nome	Identificador usado no código.
Endereço	Posição de memória associada — o <i>l-value</i> . Pode mudar entre ativações.
Tipo	Faixa de valores e operações permitidas.
Valor	Conteúdo armazenado — o <i>r-value</i> .
Tempo de vida	Intervalo em que a variável está vinculada a uma posição de memória .
Escopo	Trecho do programa em que o nome é visível .

Dois nomes diferentes vinculados ao **mesmo endereço** são **aliases** — prejudicam legibilidade e confiabilidade, o que fecha o ciclo com o capítulo 1.

5.2 Vinculação e tempo de vinculação

Vinculação é a associação entre um atributo e uma entidade. O que cai é **quando** ela acontece:

Momento	Exemplo
Projeto da linguagem	O símbolo <code>*</code> é vinculado à multiplicação.
Implementação da linguagem	A faixa de valores de <code>int</code> é definida pelo compilador.
Compilação	Uma variável é vinculada ao seu tipo em linguagem de tipagem estática.
Carga (load)	Variável <code>static</code> é vinculada a uma posição de memória.
Execução	Uma variável local é vinculada a um endereço na entrada do bloco; um valor é atribuído.

ESTÁTICA × DINÂMICA

Vinculação **estática** ocorre antes da execução e **não muda** durante ela. Vinculação **dinâmica** ocorre durante a execução, ou muda ao longo dela. A troca é sempre a mesma: estática dá **eficiência e detecção antecipada de erros**; dinâmica dá **flexibilidade** ao custo de checagem em execução e erros mais tardios.

5.3 Categorias por tempo de vida

Categoria	Alocação e vida	Exemplo	Troca principal
Estática	Vinculada antes da execução , permanece a vida inteira do programa.	<code>static</code> , variável global	eficiência ↔ flexibilidade (sem recursão, memória sempre ocupada)
Dinâmica da pilha	Vinculada na entrada do bloco , liberada no retorno.	variável local comum	permite recursão ↔ overhead de alocação por ativação
Heap explícita	Alocada e liberada por comando explícito ou referência.	<code>new</code> / <code>delete</code> , objeto dinâmico	estrutura flexível ↔ complexidade e vazamentos

Heap implícita	As próprias atribuições controlam alocação.	variável de linguagem dinâmica	máxima flexibilidade ↔ custo e perda de detecção de erros
-----------------------	--	--------------------------------	---

CASO-CHAVE QUE O PROFESSOR DESTACOU

Variável local comum = dinâmica da pilha. E o ponto fino: **o endereço pode mudar entre execuções**, porque cada ativação da função recebe um **novo registro de ativação** na pilha. Por isso recursão funciona — cada chamada tem sua própria cópia.

5.4 Escopo

Escopo responde: **"qual declaração este nome encontra?"**

	Escopo estático (léxico)	Escopo dinâmico
Determinado por	A estrutura textual do programa — dá para resolver lendo o código.	A sequência de chamadas em execução.
Busca	local → pai léxico → ancestrais, até o escopo global	local → chamador ativo → cadeia de chamadas
Quando se resolve	Em compilação	Em execução
Efeito	Mais legível e verificável	Mais flexível, muito mais difícil de ler; praticamente abandonado

Ambiente de referenciamento: o conjunto de **todas** as variáveis visíveis em uma sentença, segundo a regra de escopo em vigor.

Ocultamento: a declaração mais próxima pode **esconder** outra de mesmo nome em escopo mais externo.

A DISTINÇÃO QUE MAIS CAI

Escopo ≠ tempo de vida. Escopo trata de **visibilidade** do nome; tempo de vida trata do **vínculo com uma posição de memória**. Uma variável `static` dentro de uma função tem escopo **local** (só é visível ali) e tempo de vida **do programa inteiro** (mantém o valor entre chamadas). São eixos independentes.

5.5 Exemplo trabalhado: prever a saída

```
class Painel {
    static int nivel = 8;

    static void exibir() {
        int nivel = 3;
        System.out.print(nivel + " ");
    }

    public static void main(String[] a) {
        exibir();
        System.out.println(nivel);
    }
}
```

Saída: 3 8

- **Dentro de `exibir`** : a local `nivel = 3` **oculta** o atributo estático da classe. Pela regra de escopo estático, a busca começa no bloco mais interno e para na primeira declaração encontrada — imprime **3**.

- **De volta à `main`**: a local de `exibir` já saiu de escopo (e da pilha). A busca encontra o atributo estático — imprime **8**.

Repare que o exemplo junta os três conceitos do capítulo: **escopo estático** (a resolução é textual), **ocultamento** (a local esconde a da classe) e **tempo de vida** (a local é dinâmica da pilha e morre no retorno; a estática sobrevive).

6. Armadilhas que eu preciso evitar

Erro fácil de cometer	O que eu tenho que lembrar
Dizer que lexema e token são a mesma coisa	Lexema é a ocorrência concreta; token é a categoria. <code>soma</code> é lexema, <code>IDENT</code> é token.
Chamar forma sentencial de sentença	Sentença só tem terminais . Se ainda há não terminal, a derivação não acabou.
Confundir escopo com tempo de vida	Visibilidade do nome × vínculo com memória. <code>static</code> <code>local</code> prova que são independentes.
Responder "híbrida" quando o enunciado descreve JIT	Se houver compilação durante a execução com reuso , é JIT.
Dizer que ambiguidade é erro de sintaxe	É propriedade da gramática : a sentença é válida, mas admite duas árvores e dois significados.
Tratar a evolução das linguagens como cronologia	O que vale é domínio → contribuição → herança. Coexistência, não substituição.
Achar que recursão à esquerda quebra qualquer parser	Quebra o descendente . O LR lida bem — e até se beneficia dela.
Afirmar que uma linguagem é "a melhor"	Sempre contextualizar por domínio e por qual critério pesa mais ali.
Dizer que semântica estática é sobre significado	Apesar do nome, trata de restrições verificáveis sem executar — mais forma do que significado.
Esquecer a terceira parte da resposta	Conceito + mecanismo + consequência . A consequência é o que costuma faltar.

7. Autoteste

Respondo em voz alta antes de conferir. Se travar, volto ao capítulo correspondente.

1. Quais são os quatro critérios de avaliação e um trade-off entre eles?

Legibilidade, capacidade de escrita, confiabilidade e custo. Trade-off: mais checagens de tipo em execução elevam a confiabilidade e também o custo de execução.

2. Como eu reconheço JIT em um enunciado?

Pela compilação **durante** a execução dos trechos frequentes, com **reuso** do código gerado. Parte de uma forma intermediária, como a híbrida, mas não reinterpreta o que já compilou.

3. Por que von Neumann explica o predomínio das linguagens imperativas?

A arquitetura separa memória e processador e executa sequencialmente; isso favorece variáveis, atribuição e iteração — exatamente o núcleo do modelo imperativo.

4. Relacione Fortran, Lisp, COBOL e Smalltalk a domínio e contribuição.

Fortran / científico / eficiência numérica e primeira de alto nível viável. Lisp / simbólico e IA / funções, listas e tradição funcional. COBOL / comercial / dados de negócio e sintaxe legível. Smalltalk / OO / objetos, mensagens e ambiente integrado.

5. Qual a diferença entre forma sentencial e sentença?

Forma sentencial pode conter não terminais; sentença é composta apenas por terminais. A derivação termina ao chegar na sentença.

6. O que torna uma gramática ambígua e como se resolve?

Uma mesma sentença admitir duas árvores de análise distintas. Resolve-se estruturando a gramática em níveis (`expr` , `termo` , `fator`) que codificam precedência e associatividade.

7. Cite as três abordagens de semântica e o que cada uma usa como base.

Operacional (mudanças de estado em uma máquina abstrata), denotacional (funções e objetos matemáticos) e axiomática (asserções com pré e pós-condições).

8. Qual a divisão de trabalho entre analisador léxico e parser?

O léxico converte caracteres em tokens com base em padrões; o parser verifica se a sequência de tokens pertence à estrutura da gramática. Separar dá simplicidade, eficiência e portabilidade.

9. Por que recursão à esquerda quebra o parser descendente e como se corrige?

Porque `E` chama `E` antes de consumir entrada — nada muda entre as chamadas e a recursão é infinita. Corrige-se reescrevendo: `E → T E'` e `E' → + T E' | ε`.

10. O que ACTION e GOTO decidem em um parser LR?

ACTION[estado, terminal] decide shift, reduce, accept ou erro. GOTO[estado, não terminal] dá o novo estado após uma redução.

11. Quais são os seis atributos de uma variável?

Nome, endereço, tipo, valor, tempo de vida e escopo.

12. Em que categoria de armazenamento cai uma variável local comum, e por que o endereço pode mudar?

Dinâmica da pilha. Cada ativação da função recebe um novo registro de ativação, então o endereço varia de chamada para chamada — é isso que viabiliza a recursão.

13. Escopo e tempo de vida são a mesma coisa? Dê o contraexemplo.

Não. Uma variável `static` declarada dentro de uma função tem escopo local e tempo de vida igual ao do programa.

14. No exemplo da classe `Painel`, qual a saída e por quê?

3 8. Dentro de `exibir` a local oculta o atributo estático; de volta à `main`, a local já saiu de escopo e a busca encontra o estático.

8. Checklist final de 10 minutos

☐	Defino os quatro critérios de avaliação sem confundi-los e cito um trade-off.
☐	Reconheço JIT pela compilação durante a execução com reúso.
☐	Relaciono quatro linguagens aos seus domínios e contribuições.
☐	Explico a evolução como coexistência, não como substituição.
☐	Faço uma derivação completa e separo terminal de não terminal.
☐	Explico ambiguidade e sei desambiguar com níveis de precedência.
☐	Distingo as três abordagens de semântica e estática × dinâmica.

☐	Classifico lexemas em tokens e explico léxico × parser.
☐	Comparo descendente × ascendente e explico ACTION e GOTO.
☐	Explico por que recursão à esquerda quebra o descendente e sei reescrevê-la.
☐	Classifico variável local como dinâmica da pilha e justifico a troca de endereço.
☐	Separo escopo de tempo de vida com o contraexemplo do <code>static</code> .
☐	Prevejo a saída de um programa com ocultamento em escopo estático.

ÚLTIMA CONFERÊNCIA

Explicar em voz alta. Onde eu travar, volto ao capítulo correspondente deste guia e ao slide indicado no material do professor.